

DOM_v2

Leaf Canopy Radiative Transfer — Code Guide

This document explains every file in the DOM_v2 solver, what each one does, the physics behind it, and how the pieces connect. It is written to be readable without opening the code.

File Overview

The solver is split into eight files. **config.py** and **main.py** are unnumbered because they sit outside the physics pipeline. The six numbered files run in order:

File	Role
<code>config.py</code>	All user-facing parameters — the only file you edit for a normal run.
<code>main.py</code>	Orchestrates the six steps in order; prints results to the console.
<code>step1_quadrature.py</code>	Builds the angular grid (directions + weights) used by every other step.
<code>step2_phase.py</code>	Calculates how strongly light scatters from one direction into another.
<code>step3_uncollided.py</code>	Tracks photons that have never hit a leaf (direct beam + sky + ground bounce).
<code>step4_collided.py</code>	Iteratively solves for photons that have scattered one or more times.
<code>step5_energy.py</code>	Adds everything up and checks that energy is conserved.
<code>step6_brf.py</code>	Computes the Bi-directional Reflectance Factor at chosen view angles.

config.py — All User Parameters

This is the **only file you need to edit** for a normal run. Every other module imports from it. Changing a value here automatically propagates to all steps.

Spectral Parameters

Parameter	Default	Description
rho_L	0.06	Leaf single-sided reflectance . RED band ≈ 0.06 ; NIR band ≈ 0.45 – 0.53 . This is the fraction of incident light bounced back off one face of a leaf.
tau_L	0.04	Leaf single-sided transmittance . RED ≈ 0.04 ; NIR ≈ 0.45 . Fraction of light passing straight through the leaf.
rho_g	0.10	Lambertian ground reflectance . Fraction of downwelling flux reflected back up by the soil.

Illumination Geometry

Parameter	Default	Description
theta_solar	140.0	Solar polar angle measured from the upward zenith (degrees). Must be in $(90^\circ, 180^\circ]$. 180° = sun directly overhead; 140° = 40° above horizon. <i>Note: this is NOT the standard solar zenith angle from nadir — it is its supplement. A solar zenith of 40° means $\theta_{\text{solar}} = 140^\circ$.</i>
phi_solar	0.0	Solar azimuth angle (degrees). Defines the horizontal direction the sun comes from. 0° is the reference; set this to place the sun in a specific azimuth plane.
f_dir	0.70	Fraction of F_{in} that arrives as a direct collimated beam . The remaining $(1 - f_{\text{dir}})$ is treated as isotropic diffuse sky radiation. Clear sky ≈ 0.7 – 0.9 ; overcast ≈ 0.0 .
F_in	1.0	Total downwelling irradiance at the top of the canopy [W m^{-2}]. All results scale linearly with this value. Keeping it at 1.0 gives results as fractions of incoming.

Canopy Parameters

Parameter	Default	Description
LAI	1.5	Leaf Area Index — total one-sided leaf area per unit ground area [$\text{m}^2 \text{ m}^{-2}$]. LAI = 1 means the canopy has as much leaf area as the ground beneath it. Typical values: sparse grass ≈ 0.5 , dense crop ≈ 4 – 6 , forest ≈ 5 – 8 .

Numerical Resolution

Parameter	Default	Description
-----------	---------	-------------

N	16	Number of polar quadrature directions on the full sphere. Must be even. Half point downward ($n = 1 \dots N/2$), half upward ($n = N/2+1 \dots N$). Larger N = more accurate angular integration but slower.
M	16	Number of azimuthal directions . Uniformly spaced around 360°. Larger M captures azimuthal asymmetry better.
K	50	Number of canopy layers . The canopy (depth 0 to LAI) is divided into K equal slabs. Larger K improves vertical resolution.

Solver Convergence

Parameter	Default	Description
TOL	0.01	Relative convergence tolerance . Iteration stops when the largest relative change in the scalar irradiance profile (and boundary fluxes) drops below this value. 0.01 = 1 %. Tighter values (e.g. 0.001) give more accurate results but require more iterations, especially in NIR.
MAX_ITER	500	Safety cap on the number of source iterations. If the solver has not converged by this point a warning is printed. 500 is sufficient for all normal cases.

Method Flags

Parameter	Default	Description
QUADRATURE	"gauss_legendre"	Which angular quadrature rule to use. Options: gauss_legendre — standard Gauss-Legendre (default, good all-round); double_gauss — separate Gauss-Legendre on each hemisphere, better hemisphere flux accuracy (used in DISORT); gauss_lobatto — includes the exact nadir/zenith directions; uniform — evenly spaced, for debugging only.
SWEEP	"diamond_diff"	Which finite-difference scheme to use for the transport sweep. diamond_diff ($\alpha = 0.5$) — second-order accurate, may occasionally produce small negative intensities that are clipped to zero; step ($\alpha = 1.0$) — first-order upwind, always non-negative but less accurate.
ALPHA_DD	0.5	The diamond-difference weighting parameter. Overridden automatically when SWEEP = 'step' (forces $\alpha = 1.0$). Only change this if you want an intermediate value between DD and Step.

main.py — Orchestrator

main.py does not contain any physics. Its job is to call the six steps in the correct order, passing outputs from one step as inputs to the next, and print a summary to the console.

Execution Flow

1	Read all parameters from <code>config.py</code> .
2	Compute derived quantities: $\omega_L = \rho_L + \tau_L$, $dL = LAI / K$, $\mu_{solar} = \cos(\theta_{solar})$, etc.
3	Call <code>step1_quadrature</code> → get polar cosines (μ), azimuths (ϕ), and weights.
4	Call <code>step2_phase</code> → precompute the full scattering phase table (G_{qq}) and the solar phase column (G_{sol}). This is the most expensive setup step.
5	Call <code>step3_uncollided</code> → solve the never-scattered field analytically. Returns intensities, irradiance profiles, and the First Collision Source Q .
6	Call <code>step4_collided</code> → iterate the scattered field until convergence. Returns the converged collided intensity IC and irradiance profiles.
7	Call <code>step5_energy</code> → sum everything, compute leaf absorption, and verify energy conservation.
8	Loop over view angles and call <code>step6_brf</code> for each → print BRF values.

Key derived quantities computed in main.py:

- $\omega_L = \rho_L + \tau_L$ — single-scattering albedo: probability a photon hitting a leaf is scattered (not absorbed).
- $G = 0.5$ — extinction coefficient for a uniform leaf angle distribution (hardcoded).
- $dL = LAI / K$ — thickness of each canopy layer in LAI units.
- $\mu_{solar} = \cos(\theta_{solar})$ — polar cosine of the solar direction; negative because the sun travels *downward*.

Step 1 — step1_quadrature.py

The DOM integrates the radiative transfer equation over all directions on the sphere. It cannot do this continuously, so it approximates the integral as a weighted sum over a finite set of discrete directions. This is called **numerical quadrature**. This step builds that set of directions and the associated weights.

What it produces

- **mu** [array, length N+1] — polar cosines of the N directions, stored 1-based. Values run from -1 (straight down) to $+1$ (straight up). Indices $1 \dots N/2$ are downward ($\mu < 0$); indices $N/2+1 \dots N$ are upward ($\mu > 0$).
- **w_mu** [array, length N+1] — weight for each polar direction. Weights sum to 2.0 (the integral of 1 over $[-1, 1]$).
- **phi** [array, length M+1] — azimuth angles in radians, uniformly spaced from 0 to 2π .
- **w_phi** [scalar] — uniform azimuth weight = $2\pi / M$.

Quadrature methods

Parameter	Default	Description
gauss_legendre	default	Places directions at zeros of the Legendre polynomial P_N . Exact for polynomials up to degree $2N-1$. Best general-purpose choice.
double_gauss		Applies separate Gauss-Legendre quadrature independently to each hemisphere $[0,1]$ and $[-1,0]$. Gives better accuracy for hemisphere-integrated fluxes at the same N. Used in the DISORT code.
gauss_lobatto		Forces directions to include exactly $\mu = \pm 1$ (nadir and zenith). Slightly less accurate than gauss_legendre but useful when you need exact values at the poles.
uniform		Evenly spaced directions with equal weights. Only first-order accurate. Use for debugging or visual comparison — not for production results.

Note: The azimuthal quadrature is always uniform (rectangle rule). This converges exponentially for smooth periodic integrands.

Step 2 — step2_phase.py

When a photon hits a leaf it is either absorbed or scattered. The **phase function** $\Gamma(i \rightarrow j)$ describes how strongly light arriving from direction i is redirected into direction j . This step computes Γ for every pair of quadrature directions and stores the results in lookup tables so later steps can reuse them without recalculating.

The Bi-Lambertian Phase Function

Leaves are modelled as **bi-Lambertian** scatterers: each face reflects and transmits diffusely (equal brightness in all directions). The formula for Γ uses the scattering angle β between directions i and j :

$$\cos \beta = \mu_i \mu_j + \sqrt{(1-\mu_i^2)(1-\mu_j^2)} \cdot \cos(\phi_i - \phi_j)$$

$$\Gamma(i \rightarrow j) = (\omega_L / 3\pi) \cdot (\sin \beta - \beta \cdot \cos \beta) + (\tau_L / 3) \cdot \cos \beta$$

- The **first term** represents diffuse reflection from both leaf faces.
- The **second term** represents forward transmission through the leaf (stronger when β is small, i.e. light continues in roughly the same direction).

What it produces

- **G_qq** [4-D array, (N+1, M+1, N+1, M+1)] — the full Gauss-to-Gauss phase table. $G_{qq}[n,m,i,j] = \Gamma$ from direction (μ_i, ϕ_i) into direction (μ_j, ϕ_j) . Computed once; used in every scattering iteration.
- **G_sol** [2-D array, (N+1, M+1)] — the solar-to-Gauss phase column. $G_{sol}[i,j] = \Gamma$ from the solar direction into quadrature direction (μ_j, ϕ_j) . Used to compute how the direct beam seeds the scattered field.

This is the most computationally expensive setup step: $(N \cdot M)^2 = 65,536$ evaluations for $N=M=16$. It is computed once per spectral band.

Step 3 — step3_uncollided.py

The **uncollided field** consists of photons that have never interacted with any leaf — they travel in straight lines governed only by Beer-Lambert extinction. Because there is no scattering, this field can be solved analytically (no iteration needed).

Four sub-steps

1	Direct solar beam (downward). The beam intensity decays exponentially layer by layer: $I_{\text{beam}} \times \exp(-G \cdot dL / \mu_{\text{sun}})$ per layer. Direct downward irradiance $Fd_{\text{dir}} = \mu_{\text{sun}} \times I_{\text{beam_at_each_level}}$.
2	Diffuse sky radiation (downward). The sky is modelled as isotropic radiance $I_{\text{sky}} = (1 - f_{\text{dir}}) \cdot F_{\text{in}} / \pi$. Each downward quadrature direction n decays independently: same Beer-Lambert recurrence with its own $ \mu_n $.
3	Ground boundary condition. The total downward flux hitting the ground (direct + diffuse) is reflected by the Lambertian soil: $I_{\text{ground}} = (\rho_g / \pi) \times F_{\text{down_at_ground}}$. This becomes the lower boundary condition for the upward sweep.
4	Upward uncollided field. Each upward quadrature direction sweeps from ground to canopy top, again decaying by Beer-Lambert as it travels upward through the leaves.
5	First Collision Source (Q). $Q[i,j,k]$ is the rate at which uncollided photons scatter into direction (μ_n, ϕ_n) in layer k for the first time. It has three parts: scattering from the direct beam (Q1), from the diffuse sky (Q2), and from the upwelling ground reflection (Q3). Q is the 'seed' that drives the collided solver in Step 4.

Outputs

Inputs	Outputs
<code>mu, phi, w_mu, w_phi</code> (from Step 1)	$I0_{\text{dir}}(K+2,)$ – direct beam scalar intensity at layer edges
<code>N, M, K</code> (from config)	$I0(N+1,M+1,K+2)$ – diffuse uncollided intensity at edges
<code>solar_mu, f_dir, F_in, rho_g</code> (from config)	$Fd_{\text{dir}}(K+2,)$ – direct downward irradiance profile
<code>G, dL</code> (derived in main)	$Fd_{\text{dif}}(K+2,)$ – diffuse downward irradiance profile
<code>G_qq, G_sol</code> (from Step 2)	$Fu_{\text{unc}}(K+2,)$ – upward uncollided irradiance profile
	$Q(N+1,M+1,K)$ – First Collision Source at cell centres

Step 4 — step4_collided.py

The **collided field** consists of photons that have scattered at least once. Unlike the uncollided field, scattered photons depend on each other (a photon in direction A can scatter into direction B which then scatters into direction C, and so on). This circular dependence means the field must be found by **iteration**.

Source Iteration Algorithm

1	Initialise: set IC = 0 everywhere. Set multiple-scatter source S = 0.
2	Total source: $J = Q + S$ (first-collision seed + current multiple-scatter estimate).
3	Downward sweep through all K layers for each downward direction ($\mu < 0$). Upper boundary condition: IC = 0 at the canopy top (no scattered light enters from above). The Diamond-Difference recurrence propagates intensity layer by layer.
4	Ground boundary condition: compute how much downward collided flux hits the ground; reflect it as Lambertian upwelling.
5	Upward sweep from ground to canopy top for each upward direction ($\mu > 0$). Lower boundary condition from the ground reflection just computed.
6	Update multiple-scatter source: $S_{\text{new}}[i,j,k] = (1/\pi) \sum_{n,m} w_{\mu} \cdot w_{\phi} \cdot G_{\text{qq}}[n,m,i,j] \cdot IC_c[n,m,k]$. This says: the source in direction (i,j) at layer k equals the sum of all intensities in every other direction, weighted by the phase function.
7	Dual convergence check — both criteria must pass simultaneously: (A) max relative change in IC at the top and bottom boundaries for directions with $ \mu \geq \cos(70^\circ)$; (B) max relative change in the scalar irradiance profile over all K layers. If either fails, go back to step 2 with $S = S_{\text{new}}$.

Diamond-Difference Sweep

The Diamond-Difference (DD) scheme approximates the intensity at the centre of each layer as the average of the two surrounding edge values. For a downward direction the recurrence is:

$$IC[k+1] = a \cdot IC[k] - b \cdot J[k+\frac{1}{2}]$$

$$\text{where } a = (1 + (1-\alpha) \cdot f) / (1 - \alpha \cdot f), \quad b = f / (G \cdot (1 - \alpha \cdot f)), \quad f = G \cdot dL / \mu$$

$\alpha = 0.5$ gives second-order accuracy (Diamond Difference). $\alpha = 1.0$ gives first-order accuracy (Step method, always non-negative). Negative values that appear are clipped to zero.

Why Dual Convergence Matters

Using only the interior scalar irradiance criterion (criterion B alone) can cause the solver to stop too early in high-albedo cases like NIR, leading to energy imbalances of –40% or worse. Criterion A adds a pointwise check on the top and bottom boundary fluxes (the quantities you actually report: reflectance and transmittance) to ensure those have also stabilised.

Outputs

Inputs	Outputs
$\mu, \phi, w_{\mu}, w_{\phi}$ (from Step 1)	IC (N+1,M+1,K+2) – converged collided intensity at layer edges

N, M, K (from config)	Fd_col (K+2,) – downward collided irradiance profile
G, dL, rho_g, omega_L (derived / config)	Fu_col (K+2,) – upward collided irradiance profile
Q (N+1,M+1,K) – First Collision Source from Step 3	
G_qq (N+1,M+1,N+1,M+1) – phase table from Step 2	
tol, max_iter, alpha (from config / main)	

Step 5 — step5_energy.py

With both the uncollided and collided fields solved, this step adds all flux components together and verifies that energy is conserved. The fundamental identity is:

$$F_{in} = F_{ref} + A_{leaves} + F_{abs_ground}$$

- **F_{in}** — total incoming irradiance at canopy top (= 1.0 in normalised runs).
- **F_{ref}** — total upward flux escaping at the canopy top (= $Fu_{unc}[1] + Fu_{col}[1]$). This is the hemispherical reflectance.
- **A_{leaves}** — energy absorbed by leaves = $G \cdot (1 - \omega_L) \cdot dL \times \sum SI[k]$, where $SI[k]$ is the scalar irradiance (total radiation density) in layer k.
- **F_{abs_ground}** = $(1 - \rho_g) \times F_{trans}$. Of the flux reaching the ground, fraction ρ_g bounces back; the rest $(1 - \rho_g)$ is absorbed by the soil.

Scalar Irradiance vs. Flux

There are two ways to weight the angular integral of intensity. **Flux (irradiance)** weights by $|\mu| = |\cos \theta|$, which accounts for the projected area of a surface. **Scalar irradiance** uses no cosine weight and measures the total energy density at a point, which is what drives absorption.

$$\text{Flux: } F = \sum w_{\phi} \cdot |\mu| \cdot I(n, m)$$

$$\text{Scalar irradiance: } SI = \sum w_{\phi} \cdot I(n, m) \text{ (no } |\mu| \text{ factor)}$$

Outputs (printed + returned as dict)

Inputs	Outputs
$I0_{dir}, I0, IC$ – all intensity fields	F_{ref} – reflected flux at canopy top
$Fd_{dir}, Fd_{dif}, Fu_{unc}, Fd_{col}, Fu_{col}$ – irradiance profiles	A_{leaves} – leaf absorption
$G, dL, \omega_L, \rho_g, F_{in}$ – physical parameters	F_{trans} – total downward flux at ground
	F_{abs_g} – ground absorption
	error_pct – energy balance error (%)
	Fd_{tot}, Fu_{tot}, SI – full profiles for further analysis

Step 6 — step6_brf.py

The **Bi-directional Reflectance Factor (BRF)** is the ratio of radiance leaving the canopy in a specific direction to the radiance that a perfectly white Lambertian surface would reflect under the same illumination. It describes what a sensor or camera pointed at the canopy would measure from a given view angle.

The Gauss directions used in Steps 1–5 are fixed by the quadrature rule and almost never coincide with the desired view direction. Step 6 therefore performs a separate Diamond-Difference sweep along the requested view ray, using the already-converged intensity fields as the source.

Algorithm

1	Convert view angle: view zenith θ_v and azimuth $\phi_v \rightarrow \mu_v = \cos(\theta_v)$, ϕ_v in radians.
2	Build view-ray source $J_v[k]$ at each cell centre: $J_v = \Gamma(\text{sun} \rightarrow \text{view}) \cdot I0_dir_c + \sum (w/\pi) \cdot \Gamma(n, m \rightarrow \text{view}) \cdot (I0_c + IC_c)$. This says: the source for the view ray equals all radiation (uncollided + collided) in every Gauss direction, weighted by how strongly each direction scatters into the view direction.
3	Ground boundary condition: compute total downward flux at ground (all components including direct beam); reflect Lambertian $\rightarrow I_v[K+1] = \rho_g/\pi \times F_down$.
4	Upward Diamond-Difference sweep from ground to top along μ_v , using J_v as the source. This is a single-pass sweep (no iteration needed because J_v is already known from the converged fields).
5	BRF = $\pi \cdot I_v[1] / (\mu_v \cdot F_in)$. Dividing by $\mu_v \cdot F_in$ normalises by the equivalent Lambertian reflectance.

Outputs

Inputs	Outputs
view_zenith_deg, view_azimuth_deg (from main loop)	BRF (float) – dimensionless bidirectional reflectance factor
I0, IC, I0_dir – converged intensity fields	
mu, phi, w_mu, w_phi, G_sol – quadrature + solar phase	
omega_L, tau_L, rho_g, F_in, alpha – physical params	

The function is called once per view angle. In main.py it is evaluated at zenith angles 10°, 30°, 45°, and 60° in the backscatter plane ($\phi_v = 180^\circ$).

Quick-Start: Changing Spectral Band

To switch between RED and NIR, edit only the Spectral Parameters section of config.py:

```
# RED band
rho_L = 0.06 # leaf reflectance
tau_L = 0.04 # leaf transmittance
rho_g = 0.10 # ground reflectance

# NIR band
rho_L = 0.45
tau_L = 0.45
rho_g = 0.15
```

Run `python main.py` after each change. No other file needs to be modified.

Quick-Start: Changing LAI

Edit only LAI in config.py, e.g.:

```
LAI = 4.0
```

The layer thickness $dL = LAI / K$ is recomputed automatically in main.py.

Data Flow Summary

The table below shows which variables flow between steps.

Produced by	Variable	Used by
config.py	rho_L, tau_L, rho_g, LAI, ...	main.py (all steps)
Step 1	mu, w_mu, phi, w_phi	Steps 2, 3, 4, 5, 6
Step 2	G_qq	Steps 3, 4, 6
Step 2	G_sol	Steps 3, 6
Step 3	I0_dir, I0, Fd_dir, Fd_dif, Fu_unc	Steps 5, 6
Step 3	Q (First Collision Source)	Step 4
Step 4	IC, Fd_col, Fu_col	Steps 5, 6
Step 5	F_ref, A_leaves, F_abs_g, error_pct	Printed / returned
Step 6	BRF (one value per view angle)	Printed / returned